

Dynamisch Docker-omgevingbeheer voor CTF-events op on-prem infrastructuur.

Dynamisch Docker-omgevingbeheer voor CTF-events.

Manu Devloo.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. T. Clauwaert

Co-promotor: Dhr. L. Singier

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Deze bachelorproef werd uitgevoerd in het kader van de opleiding Professionele bachelor toegepaste informatica aan HOGENT. Mijn interesse in cybersecurity, naast mijn dagelijkse bezigheden binnen softwareontwikkeling, heeft ertoe geleid dat ik dit onderwerp heb gekozen. Binnen dit onderzoek werd gezocht naar een aanpak die zowel veilig als praktisch inzetbaar is op eigen hardware.

Graag bedank ik mijn promotor, dhr. Thomas Clauwaert, en mijn co-promotor, dhr. Laurens Singier, voor hun begeleiding, feedback en kritische vragen tijdens dit traject.

Samenvatting

Capture The Flag (CTF)-events zijn cybersecuritywedstrijden waarbij deelnemers technische uitdagingen oplossen om punten te behalen. Een deel van deze uitdagingen vereist een netwerktoegankelijke service, bijvoorbeeld een webapplicatie die aangevallen moet worden. Op on-premise infrastructuur worden dergelijke challenge-omgevingen vaak manueel of met ad-hoc scripts beheerd. Dit verhoogt de operationele belasting en vergroot het risico op misconfiguraties, onvoldoende isolatie en resource-uitputting op de host.

Deze bachelorproef onderzoekt hoe Docker-containers die gekoppeld zijn aan CTFd-challenges dynamisch kunnen worden beheerd, aangemaakt en beëindigd op basis van gebruikersacties en configureerbare parameters binnen een volledig self-hosted context. De centrale doelstelling is een schaalbare aanpak uit te werken die veilige isolatie en afdwingbare resource-limieten combineert met reproduceerbaar beheer, zonder afhankelijk te zijn van betaalde diensten.

De methodologie combineert een literatuurstudie met praktijkgericht onderzoek. In de literatuurstudie worden direct toepasbare best practices rond containerbeveiliging, netwerkisolatie en resourcebeheer geïnventariseerd, evenals de integratiemogelijkheden via de Docker Engine API. Vervolgens wordt een proof-of-concept gerealiseerd in de vorm van een CTFd-plugin die de containerlevenscyclus automatiseert (opstart, beschikbaar stellen, timeouts en opruimen) en beleidsregels afdwingt.

De oplossing wordt gevalideerd met een testplan dat functionele tests, beveiligingstests (onder meer container-escape pogingen en resource-exhaustion scenario's) en load tests omvat. Het eindresultaat bestaat uit de proof-of-concept, documentatie en aanbevelingen voor het opzetten en beheren van CTF-infrastructuur op eigen hardware.

Inhoudsopgave

Lijst van figuren	viii
Lijst van tabellen	ix
Lijst van codefragmenten	x
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	2
1.4 Scope en afbakening	2
1.5 Opzet van deze bachelorproef	3
2 Achtergrond en begrippenkader	4
2.1 Capture The Flag (CTF)	4
2.1.1 Types challenges	4
2.1.2 Deployment van challenges: statisch vs. dynamisch	5
2.1.3 Bekende CTF-platformen	5
2.2 CTFd als platform	5
2.2.1 Architectuur van CTFd	5
2.2.2 Het plugin-systeem	5
2.2.3 Challenge-beheer en granulariteit	6
2.3 Docker en containerisatie	6
2.3.1 Wat is Docker?	6
2.3.2 Docker Engine API	6
2.3.3 Docker SDK voor Python	6
2.3.4 Docker Compose	6
2.3.5 Resource management	6
2.3.6 Netwerkisolatie	7
2.4 Beveiliging van containers	7
2.4.1 Beveiligingsrisico's bij dynamisch aanmaken van containers	7
2.4.2 Direct toepasbare basismaatregelen	7
2.4.3 Container escape	8
2.4.4 Resource exhaustion en DoS-risico's	8

2.5	Load balancing zonder Kubernetes	8
2.5.1	Waarom geen Kubernetes?	8
2.5.2	Alternatieven	8
2.5.3	Wanneer is schaling nodig?	8
3	Methodologie	9
4	Huidige situatie	10
4.1	Organisatie van CTF-events op eigen infrastructuur	10
4.2	Manuele container deployments	10
4.3	Overzicht van bestaande tools en hun tekortkomingen	11
4.4	Conclusie: wat ontbreekt er?	11
5	Gewenste situatie	12
5.1	Vereisten voor de oplossing	12
5.1.1	Functionele vereisten	12
5.1.2	Niet-functionele vereisten	13
5.2	Architectuurvisie	13
5.2.1	Componenten en verantwoordelijkheden	13
5.2.2	Datastromen op hoofdlijnen	14
5.3	Granulariteitsbeslissing: container per gebruiker of per team	14
5.4	Schaalbaarheidsvereisten	14
6	Proof-of-Concept	16
6.1	Architectuur van de PoC	16
6.2	Implementatie van de CTFd-plugin	16
6.2.1	Technische keuzes	16
6.2.2	Container lifecycle management	16
6.2.3	Configuratie van een challenge-container	16
6.2.4	Gebruikersinterface in CTFd	16
6.3	Beveiliging in de PoC	16
6.3.1	Isolatie tussen containers van verschillende teams/gebruikers	16
6.4	Schaalbaarheid en load balancing	16
6.5	Automatisatie en gebruiksgemak voor organisatoren	16
7	Validatie en testing	17
7.1	Testplan	17
7.1.1	Doelstellingen	17
7.1.2	Testomgeving	17
7.2	Functionele tests	17
7.2.1	Aanmaken en beëindigen van containers	17
7.2.2	Isolatie tussen gebruikers	17

7.3	Beveiligingstests	17
7.3.1	Container escape pogingen	17
7.3.2	Resource exhaustion simulaties	17
7.4	Load testing	17
7.4.1	Testopzet	17
7.4.2	Meetpunten	17
7.4.3	Resultaten en conclusies	17
8	Conclusie	18
8.1	Beantwoording van de onderzoeksvragen	18
8.2	Beperkingen van de PoC	18
8.3	Aanbevelingen voor productiegebruik	18
8.4	Mogelijke uitbreidingen	18
A	Onderzoeksvoorstel	19
A.1	Introductie	19
A.1.1	Probleemstelling	19
A.1.2	Onderzoeksvraag	20
A.1.3	Onderzoeksdoelstelling	21
A.2	State-of-the-art	21
A.3	Methodologie	21
A.4	Verwacht resultaat, conclusie	22
B	Installatie- en beheerinstructies	23
C	Configuratiebestanden	24
D	Testresultaten in detail	25
E	Onderzoekslog	26
	Bibliografie	28

Lijst van figuren

Lijst van tabellen

Lijst van codefragmenten

1

Inleiding

Capture The Flag (CTF)-events zijn cybersecuritywedstrijden waarbij deelnemers technische uitdagingen oplossen om punten te scoren (Innovirtuoso, [z.d.](#)). Ze worden georganiseerd door hogescholen, universiteiten, bedrijven en onafhankelijke gemeenschappen als educatieve en competitieve oefeningen. Elke deelnemer of elk team werkt aan zogenaamde *challenges*: geïsoleerde omgevingen met een kwetsbaarheid die uitgebuit moet worden om een *flag* te bemachtigen.

Naarmate CTF-events populairder worden, groeit ook de behoefte aan robuuste, schaalbare infrastructuur om dergelijke challenges betrouwbaar aan te bieden. Momenteel worden challenges op eigen infrastructuur vaak manueel of semi-automatisch beheerd, wat leidt tot operationele overhead, beveiligingsrisico's en schaalbaarheidsknelpunten (Singier, [2025](#)).

1.1. Probleemstelling

Bij het organiseren van CTF-events op eigen (on-premise) infrastructuur is het dynamisch en veilig beheren van containers voor challenges een bijzonder complexe opgave. Organisatoren moeten voor elke challenge een geïsoleerde containeromgeving opzetten, beheren en weer afbreken op basis van gebruikersacties. Dit proces verloopt vandaag grotendeels manueel of met eenvoudige scripts, wat de volgende problemen met zich meebrengt:

- Manuele interventie bij het opstarten en stopzetten van containers kost tijd en is foutgevoelig.
- Er is geen gegarandeerde isolatie tussen containers van verschillende teams of gebruikers.
- Resource-uitputting (CPU, RAM) door ongecontroleerde containers vormt een risico voor de stabiliteit van de host.

- Bestaande cloud-gebaseerde platformen (bijv. gehoste CTFd-diensten) zijn niet altijd geschikt voor organisaties die werken met privédata of op eigen hardware willen draaien.

De doelgroep van dit onderzoek zijn CTF-organisatoren en systeembeheerders die events organiseren op eigen on-premise infrastructuur en op zoek zijn naar een geautomatiseerde, veilige en schaalbare oplossing.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag luidt:

Hoe kunnen Docker-containers die gelinkt zijn aan CTFd-challenges dynamisch worden beheerd, aangemaakt en beëindigd op basis van gebruikersacties en ingestelde parameters, op on-premise infrastructuur?

Hieruit volgen de volgende deelvragen:

- Welke beveiligingsmaatregelen zijn minimaal vereist om container-escapes en misbruik te voorkomen?
- Op welke manier kunnen resource-limieten (CPU, RAM) en netwerkisolatie technisch worden afgedwongen?
- Hoe kan een CTFd-plugin de Docker Engine API aanspreken zonder zware externe afhankelijkheden?
- Hoe valideert men de stabiliteit en veiligheid van de oplossing onder zware belasting?

1.3. Onderzoeksdoelstelling

Het beoogde resultaat van deze bachelorproef is een werkende proof-of-concept (PoC) die aantoont hoe CTFd-challenges dynamisch en veilig beheerd kunnen worden op eigen infrastructuur. De PoC omvat een CTFd-plugin die communiceert met de Docker Engine API, resource-limieten afdwingt en netwerkisolatie garandeert. De oplossing is volledig self-hosted, open-source en modulair uitbreidbaar.

1.4. Scope en afbakening

Dit onderzoek beperkt zich tot on-premise, self-hosted omgevingen. Kubernetes wordt bewust niet als primaire oplossing gekozen omwille van de operationele complexiteit die niet in verhouding staat met de schaal van een doorsnee CTF-event. Cloudplatforms en betaalde diensten vallen buiten scope. De focus ligt op open-source tooling die draait op standaard Linux-servers.

1.5. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt de achtergrond en het begrippenkader geschetst: wat zijn CTF-events, hoe werkt CTfD, wat zijn de relevante Docker-concepten en welke beveiligingsaspecten spelen een rol?

In Hoofdstuk 3 wordt de onderzoeksmethodologie toegelicht.

In Hoofdstuk 4 wordt de huidige situatie (AS-IS) geanalyseerd: hoe worden CTF-events vandaag beheerd en wat zijn de tekortkomingen?

In Hoofdstuk 5 wordt de gewenste situatie (TO-BE) beschreven, inclusief de architectuurvisie en de vereisten voor de oplossing.

In Hoofdstuk 6 wordt de proof-of-concept uitgewerkt: architectuur, implementatie, beveiliging en schaalbaarheid.

In Hoofdstuk 7 worden de testresultaten besproken: functionele tests, beveiligingstests en load tests.

In Hoofdstuk 8, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen, samen met aanbevelingen voor toekomstig werk.

2

Achtergrond en begrippenkader

Dit hoofdstuk biedt de lezer alle achtergrondkennis die nodig is om de rest van de bachelorproef te begrijpen. Iemand die niet vertrouwd is met CTF-events, Docker of containerbeveiliging, beschikt na het lezen van dit hoofdstuk over een solide basis. Achtereenvolgens komen CTF-events, het CTFd-platform, Docker en containerisatie, containerbeveiliging en load balancing zonder Kubernetes aan bod.

2.1. Capture The Flag (CTF)

Een Capture The Flag (CTF)-event is een cybersecuritywedstrijd waarbij deelnemers technische uitdagingen moeten oplossen om een geheime reeks tekens, de zogenaamde *flag*, te bemachtigen en in te dienen voor punten (Innovirtuoso, [z.d.](#)). CTF-events worden ingezet als educatief middel, als rekruteringsstool en als competitieve sport binnen de cybersecuritygemeenschap.

2.1.1. Types challenges

CTF-challenges zijn doorgaans opgedeeld in categorieën:

- **Web:** kwetsbaarheden in webapplicaties zoals SQL-injectie, Cross-Site Scripting (XSS) en onveilige autorisatie.
- **Pwn (Binary Exploitation):** het exploiteren van kwetsbaarheden in gecompileerde binaire bestanden, bijv. buffer overflows.
- **Reverse Engineering:** analyse en begrip van gecompileerde software zonder broncode.
- **Cryptografie:** het kraken of omzeilen van cryptografische systemen en protocollen.

- **Forensics:** het reconstrueren van gebeurtenissen uit digitale artefacten zoals logbestanden, geheugenafbeeldingen of netwerkopnamen.

2.1.2. Deployment van challenges: statisch vs. dynamisch

Challenges kunnen op twee manieren worden uitgerold. Bij een **statische deployment** wordt één gedeelde instantie van de challenge opgezet die alle deelnemers tegelijk gebruiken. Dit is eenvoudig te beheren, maar brengt risico's mee: één deelnemer kan de omgeving voor alle anderen beïnvloeden. Bij een **dynamische deployment** krijgt elk team of elke gebruiker een eigen, geïsoleerde containerinstantie. Dit verhoogt de integriteit van de wedstrijd maar vereist automatisering (CTFd, [z.d.](#)).

2.1.3. Bekende CTF-platformen

De meest gebruikte platformen voor het organiseren van CTF-events zijn:

- **CTFd:** het meest wijdverspreide open-source platform, met ondersteuning voor plugins (CTFd LLC, [z.d.](#)).
- **rCTF:** een alternatief platform ontwikkeld door het redpwn-team (redpwn, [z.d.](#)).
- **kCTF:** een door Google ontwikkeld Kubernetes-gebaseerd platform, ontworpen voor grote-schaal events (Google, [z.d.](#)).

2.2. CTFd als platform

CTFd is een open-source CTF-platform gebouwd in Python met het Flask-framework (CTFd LLC, [z.d.](#)). Het biedt een gebruiksvriendelijke interface voor zowel organisatoren als deelnemers en is modulair uitbreidbaar via een plugin-systeem.

2.2.1. Architectuur van CTFd

CTFd bestaat uit een Flask-backend, een relationele database (standaard SQLite, in productie MySQL of PostgreSQL) en een frontend op basis van Bootstrap. De applicatie draait doorgaans achter een reverse proxy zoals Nginx. Alle functionaliteit is bereikbaar via een REST API.

2.2.2. Het plugin-systeem

CTFd ondersteunt plugins die extra functionaliteit toevoegen aan het platform. Een plugin is een Python-pakket dat in de CTFd/plugins/-map geplaatst wordt. Plugins kunnen nieuwe challenge-types registreren, routes toevoegen en bestaande logica uitbreiden (CTFd LLC, [z.d.](#)).

2.2.3. Challenge-beheer en granulariteit

CTFd ondersteunt standaard statische challenges. Voor dynamische instanties, waarbij elke gebruiker of elk team een eigen containerinstantie krijgt, zijn plugins vereist, zoals de CTFd-Whale plugin (Glzjin, [z.d.](#)). De granulariteit (container per gebruiker of per team) is een ontwerpbeslissing met impact op resource-gebruik en isolatie.

2.3. Docker en containerisatie

2.3.1. Wat is Docker?

Docker is een platform voor het bouwen, distribueren en draaien van applicaties in containers (Arxiv, [2024](#)). Een container is een geïsoleerd, lichtgewicht proces dat gebruikmaakt van Linux-kernelmechanismen (namespaces, cgroups) om te worden afgeschermd van andere processen en van de host.

Containers versus virtuele machines: waar een virtuele machine een volledig besturingssysteem emuleert, deelt een container de kernel van de host. Dit maakt containers veel sneller op te starten en minder resource-intensief, maar vergt zorgvuldigere beveiligingsconfiguratie.

2.3.2. Docker Engine API

De Docker Engine API is een RESTful HTTP-API die het mogelijk maakt om programmatisch te communiceren met de Docker-daemon (Docker Inc., [z.d.-b](#)). Via deze API kunnen containers worden aangemaakt, gestart, gestopt en verwijderd, en kunnen netwerken en volumes worden beheerd. De API is de basis voor alle Docker-tooling, inclusief de Docker CLI.

2.3.3. Docker SDK voor Python

De Docker SDK voor Python is een officiële client-bibliotheek die de Docker Engine API omhult en in Python aanroept (Docker Inc., [z.d.-c](#)). Het is de aanbevolen manier om Docker programmatisch te besturen vanuit Python-applicaties, zoals een CTFd-plugin.

2.3.4. Docker Compose

Docker Compose is een tool voor het definiëren en draaien van multi-container applicaties via een YAML-configuratiebestand (Docker Inc., [z.d.-a](#)). Voor challenge-configuraties biedt Compose een declaratieve manier om containerparameters (image, netwerk, resource-limieten) te definiëren.

2.3.5. Resource management

Docker biedt mechanismen voor het beperken van resource-gebruik per container, gebaseerd op Linux cgroups:

- CPU-limieten: `-cpus` en `-cpu-shares`
- Geheugenlimieten: `-memory` en `-memory-swap`
- I/O-limieten via `blkio-cgroup-parameters`

2.3.6. Netwerkisolatie

Docker biedt verschillende netwerkmodi:

- **Bridge:** de standaardmodus waarbij containers communiceren via een virtuele brug, maar gescheiden zijn van de host.
- **None:** geen netwerktoegang.
- **Custom overlay/bridge networks:** containers kunnen worden ingedeeld in geïsoleerde virtuele netwerken.

2.4. Beveiliging van containers

2.4.1. Beveiligingsrisico's bij dynamisch aanmaken van containers

Wanneer containers dynamisch worden aangemaakt op basis van gebruikersinvoer, ontstaan extra risico's: een kwaadaardige deelnemer kan proberen de configuratie te manipuleren om bevoorrechte toegang te krijgen of om resources te monopoliseren.

2.4.2. Direct toepasbare basismaatregelen

De OWASP Docker Security Cheat Sheet (OWASP, 2024) en artikelen over containerbeveiliging (Aikido, 2025) identificeren een reeks basismaatregelen:

- **Privilegebeheer:** containers draaien standaard als root; dit wordt bij voorkeur vermeden via `-user`.
- **Read-only bestandssysteem:** via `-read-only` worden schrijfacties beperkt.
- **Capabilities beperken:** Linux capabilities die niet nodig zijn (bijv. `NET_ADMIN`, `SYS_ADMIN`) worden verwijderd via `-cap-drop ALL`.
- **Seccomp-profielen:** beperken de systeemoproepen die een container mag uitvoeren.
- **AppArmor/SELinux:** MAC-profielen die extra toegangscontrole bieden.
- **No-new-privileges:** de optie `-security-opt no-new-privileges` voorkomt privilege-escalatie.

Het CIS Docker Benchmark (Center for Internet Security, 2023) biedt een uitgebreide checklist van aanbevolen beveiligingsinstellingen voor Docker-hosts en -containers.

2.4.3. Container escape

Een container escape is een aanval waarbij een kwaadaardige actor de isolatielaag van de container doorbreekt en toegang verkrijgt tot de host of andere containers. Bekende technieken maken gebruik van kwetsbaarheden in de Docker-daemon, misbruik van gemounte sockets (`/var/run/docker.sock`) of misconfiguraties (bijv. *privileged mode*) (National Institute of Standards and Technology, [z.d.](#)).

2.4.4. Resource exhaustion en DoS-risico's

Zonder resource-limieten kan een deelnemer de hostserver overbelasten door een excessief aantal containers te starten of door een container alle CPU- en geheugen-capaciteit te laten consumeren. Dit vormt een *Denial-of-Service* (DoS)-risico voor alle overige deelnemers.

2.5. Load balancing zonder Kubernetes

2.5.1. Waarom geen Kubernetes?

Kubernetes is het de-facto standaardplatform voor containerorkestratie op schaal (Google, [z.d.](#)). Het biedt ingebouwde load balancing, automatische schaling en zelfherstel. Voor kleinschalige CTF-events wegen de voordelen echter niet op tegen de operationele complexiteit: Kubernetes vereist een substantieel cluster, diepgaande kennis en aanzienlijke overhead voor beheer.

2.5.2. Alternatieven

- **Docker Swarm:** Docker's ingebouwde orkestratielaag voor meerdere nodes (Docker Inc., [z.d.-d](#)). Eenvoudiger dan Kubernetes, maar minder uitgebreid.
- **Traefik:** een dynamische reverse proxy die automatisch routes configureert op basis van Docker-labels (Traefik Labs, [z.d.](#)). Ideaal als *entry point* voor dynamisch aangemaakte containers.
- **HAProxy:** een betrouwbare, performante TCP/HTTP load balancer (HAProxy Technologies, [z.d.](#)) voor geavanceerdere routeringsconfiguraties.
- **Nginx:** een veelgebruikte webserver die ook als reverse proxy en load balancer kan fungeren.

2.5.3. Wanneer is schaling nodig?

Bij CTF-events varieert de belasting sterk: er zijn piekbelastingen bij de start van een event wanneer veel deelnemers tegelijk challenges starten. De infrastructuur moet deze piekbelasting kunnen opvangen zonder de stabiliteit van het platform te compromitteren.

3

Methodologie

Dit onderzoek combineert een literatuurstudie met praktijkgericht werk. Het verloopt in vier opeenvolgende fasen.

Fase 1 – Literatuurstudie en begrippenkader (Hoofdstuk 2): Via literatuurstudie worden de relevante concepten en de state-of-the-art in kaart gebracht. De focus ligt op CTFd, de Docker Engine API, containerbeveiliging en load balancing op on-premise infrastructuur.

Fase 2 – Analyse van de huidige en gewenste situatie (Hoofdstukken 4 en 5): De huidige aanpak voor het beheren van CTF-challenges op eigen infrastructuur wordt geanalyseerd (AS-IS). Op basis hiervan en de literatuurstudie worden functionele en niet-functionele vereisten opgesteld en een architectuurvisie uitgetekend (TO-BE).

Fase 3 – Proof-of-Concept (Hoofdstuk 6): Er wordt een werkende PoC ontwikkeld: een CTFd-plugin die via de Docker Engine API containers dynamisch beheert. De PoC implementeert resource-limieten, netwerkislatie en de basisbeveiligingsmaatregelen die in de literatuurstudie zijn geïdentificeerd. De schaalbaarheid wordt getest met een multi-VM opzet en een reverse proxy.

Fase 4 – Validatie en testing (Hoofdstuk 7): De PoC wordt gevalideerd aan de hand van een testplan met functionele tests, beveiligingstests (container escape pogingen, resource exhaustion) en load tests met tools zoals k6 (Grafana Labs, [z.d.](#)) of Locust (Locust contributors, [z.d.](#)).

4

Huidige situatie

Dit hoofdstuk analyseert hoe CTF-events vandaag worden georganiseerd op eigen infrastructuur. Het brengt de problemen en beperkingen van de huidige aanpak in kaart als vertrekpunt voor de gewenste oplossing.

4.1. Organisatie van CTF-events op eigen infrastructuur

Bij een on-premise CTF-opzet draait een centraal platform (zoals CTFd) voor gebruikersbeheer, challenges en scoring. Challenges die enkel uit een beschrijving en een download bestaan, vergen weinig infrastructuur. Service-gebaseerde challenges (bijv. een webapplicatie die aangevallen moet worden) vereisen echter een runtime-omgeving die voor deelnemers bereikbaar is.

In veel opzetten wordt die runtime-omgeving gescheiden van het platform beheerd: de organisator bouwt een Docker-image en start één of meerdere containers op een Docker-host. De deelnemer krijgt vervolgens een URL of een host/poortcombinatie om de challenge te bereiken. De documentatie van CTFd beschrijft dit principe expliciet: de challenge-service wordt extern uitgerold en daarna gelinkt vanuit het platform (CTFd, [z.d.](#)). Op basis van gesprekken en beschikbare interne documentatie blijkt dat deze werkwijze vaak handmatig of met beperkte automatisering wordt uitgevoerd (Singier, [2025](#)).

4.2. Manuele container deployments

Wanneer challenge-containers manueel worden uitgerold, gebeurt dit doorgaans via `docker run` of `docker compose`. Organisatoren moeten daarbij onder meer Docker-images beheren, poortmapping instellen, DNS- of reverse-proxyconfiguratie voorzien en containers opruimen na afloop. Tijdens een event, met piekbelasting en veel gelijktijdige deelnemers, zorgt dit voor operationele overhead en een verhoogde kans op fouten.

Bovendien ontbreken bij manuele uitrol vaak centrale beleidsregels. Zonder consistente standaardinstellingen voor resource-limieten en netwerkisolatie ontstaat het risico dat één of meerdere containers de host overbelasten of dat deelnemers elkaars instanties kunnen beïnvloeden. Ook het correct en tijdig verwijderen van containers (bijv. na een timeout) vereist bijkomende scripts en monitoring, wat niet in alle scenario's sluitend is.

4.3. Overzicht van bestaande tools en hun tekortkomingen

CTFd is in de basis gericht op het beheren van de wedstrijdlogica en bevat geen ingebouwde orkestratie voor het per gebruiker/team opstarten van service-instanties (CTFd LLC, [z.d.](#)). Voor dynamische deployments bestaan er community-plugins, zoals CTFd-Whale (Glzjin, [z.d.](#)), die proberen containerbeheer te integreren in het platform.

Hoewel dergelijke oplossingen een goed vertrekpunt vormen, voldoet een bestaande tool niet automatisch aan de randvoorwaarden van dit onderzoek: volledig self-hosted inzetbaar, zonder terugkerende kosten, met een voorkeur voor open-source componenten en met voldoende controle over beveiligingsinstellingen en afhankelijkheden. Een alternatief is het gebruik van eigen scripts en conventies, maar dan ontbreekt doorgaans een nauwe koppeling met gebruikersacties in CTFd en is het moeilijker om uniforme beleidsregels (isolatie, quota, timeouts) af te dwingen.

Tot slot bestaan er gehoste of cloud-gebaseerde platformen, maar die vallen buiten de scope van deze bachelorproef wanneer organisaties omwille van privacy, controle of koststructuur op eigen hardware willen blijven.

4.4. Conclusie: wat ontbreekt er?

Uit de analyse volgt dat de huidige aanpak vooral tekortschiet op het vlak van automatisering en afdwingbaarheid. Concreet ontbreekt er een mechanisme dat:

- de containerlevenscyclus rechtstreeks koppelt aan acties in CTFd (start/stop, timeouts en automatisch opruimen);
- beveiligings- en resourcebeleid standaard toepast (CPU/RAM-limieten, netwerkisolatie en veilige standaardinstellingen);
- schaalbaar omgaat met piekbelasting zonder dat organisatoren manueel moeten ingrijpen;
- beheer en monitoring vereenvoudigt (overzicht van actieve instanties en toewijzing aan gebruiker/team).

Deze ontbrekende elementen vormen de input voor de gewenste situatie (TO-BE) en de ontwerpkeuzes van de proof-of-concept.

5

Gewenste situatie

Op basis van de tekortkomingen die in het vorige hoofdstuk werden geïdentificeerd, beschrijft dit hoofdstuk de vereisten voor de oplossing en de architectuurvisie voor het gewenste systeem.

5.1. Vereisten voor de oplossing

5.1.1. Functionele vereisten

De gewenste oplossing automatiseert het beheer van service-gebaseerde challenges. Concreet moet het systeem:

- bij het starten van een challenge automatisch een containerinstantie voorzien per gebruiker of per team;
- de volledige containerlevenscyclus beheren (aanmaken, starten, stopzetten, verwijderen en opruimen na timeout);
- per challenge configureerbare resource-limieten afdwingen (CPU en RAM);
- netwerkisolatie afdwingen tussen instanties van verschillende gebruikers/teams en ongewenste laterale beweging verhinderen;
- een mechanisme voorzien om het aantal actieve instanties te limiteren (per challenge en/of globaal) om resource-uitputting te vermijden;
- organisatoren een beheerbare manier bieden om challenge-parameters te configureren (Docker-image, poorten, timeouts, limieten);
- een overzicht bieden van actieve instanties, inclusief eigenaar (gebruiker/team), status en vervaltijd, met voldoende logging voor foutopsporing.

5.1.2. Niet-functionele vereisten

Naast de functionele vereisten gelden de volgende niet-functionele vereisten:

- **Self-hosted en open-source:** de oplossing moet inzetbaar zijn op eigen infrastructuur, zonder afhankelijkheid van betaalde diensten.
- **Beveiliging:** veilige standaardinstellingen en een minimale aanvalsoppervlakte, met focus op isolatie en het vermijden van container-escapes (Center for Internet Security, [2023](#); OWASP, [2024](#)).
- **Geen Kubernetes als vereiste:** de basiswerking mag niet afhankelijk zijn van Kubernetes, om de operationele complexiteit te beperken.
- **Schaalbaarheid:** de opzet moet piekbelasting kunnen opvangen en uitbreidbaar zijn naar meerdere hosts of VM's.
- **Beheerbaarheid en reproduceerbaarheid:** installatie- en beheerinstruaties, consistente configuratie en voldoende logging/monitoring.
- **Modulariteit:** uitbreidbaar naar extra challenge-types of extra beleidsregels zonder ingrijpende aanpassingen aan de kernlogica.

5.2. Architectuurvisie

De oplossing wordt uitgewerkt als een uitbreiding op CTFd via het plugin-systeem (CTFd LLC, [z.d.](#)). De plugin fungeert als orkestratielaag die gebruikersacties (start/stop van een challenge-instantie) vertaalt naar containeracties op een Docker-host.

5.2.1. Componenten en verantwoordelijkheden

- **CTFd:** voorziet authenticatie, challenge-definities en de gebruikersinteractie.
- **CTFd-plugin:** beheert de containerlevenscyclus, houdt de toewijzing bij (gebruiker/team → container) en past beleidsregels toe (timeouts, quota, resource-limieten, isolatie).
- **Docker-host(s):** draaien de challenge-containers en worden aangestuurd via de Docker Engine API (Docker Inc., [z.d.-b](#)). De communicatie kan gebeuren via de Docker SDK voor Python (Docker Inc., [z.d.-c](#)) of via directe API-aanroepen.
- **Reverse proxy:** fungeert als toegangspunt en routeert verkeer naar dynamisch aangemaakte containers (bijv. via Traefik (Traefik Labs, [z.d.](#)) of alternatieven zoals Nginx/HAProxy (HAProxy Technologies, [z.d.](#))).

5.2.2. Datastromen op hoofdlijnen

- **Start:** een deelnemer start een challenge → de plugin valideert beleidsregels (quota/timeout) → maakt een container aan met limieten en isolatie → publiceert een toegangspunt → de deelnemer krijgt een URL of verbindingsinformatie.
- **Stop en opruimen:** bij expliciet stopzetten of bij het verlopen van de timeout worden container en gerelateerde resources (netwerk, metadata) verwijderd om resourcelekken te vermijden.

5.3. Granulariteitsbeslissing: container per gebruiker of per team

De keuze tussen een instantie per gebruiker of per team beïnvloedt zowel isolatie als resourceverbruik:

- **Per gebruiker:** maximale isolatie en minder risico op interferentie, maar het aantal containers stijgt snel bij grotere events.
- **Per team:** minder instanties en lagere infrastructuurkosten, maar teamleden delen één omgeving en kunnen elkaars voortgang beïnvloeden.

Binnen de PoC wordt uitgegaan van een flexibele aanpak: wanneer het event in teammodus verloopt, wordt een instantie per team voorzien; bij individuele deelname wordt een instantie per gebruiker voorzien. Hierdoor blijft de isolatie passend bij het wedstrijdmodel, terwijl onnodige containerexplosie wordt vermeden.

5.4. Schaalbaarheidsvereisten

De infrastructuur moet bestand zijn tegen piekbelasting, bijvoorbeeld wanneer veel deelnemers gelijktijdig een service-challenge starten. Dit vertaalt zich naar de volgende vereisten:

- **Piekbestendig opstartpad:** het systeem moet een piek aan containercreatie kunnen verwerken zonder dat het platform onbeschikbaar wordt.
- **Begrenzing en evenwichtige verdeling:** limieten op actieve instanties en afdwingbare quota vermijden dat één challenge of deelnemer disproportioneel resources consumeert.
- **Horizontale uitbreiding:** de architectuur moet uitbreidbaar zijn naar meerdere Docker-hosts. Voor taakverdeling over meerdere hosts kan Docker Swarm een mogelijke optie zijn (Docker Inc., [z.d.-d](#)), maar de PoC focust in eerste instantie op een beheerbare opzet die zonder Kubernetes functioneert.

- **Dynamische routing:** de reverse proxy moet dynamisch kunnen routeren naar nieuw aangemaakte instanties, zodat deelnemers steeds een correct toegangspunt ontvangen (Traefik Labs, [z.d.](#)).

6

Proof-of-Concept

Dit hoofdstuk beschrijft de uitwerking van de proof-of-concept (PoC). Na de architectuurbespreking komen de implementatiedetails van de CTFd-plugin, de beveiligingsmaatregelen, de schaalbaarheidsopzet en het gebruiksgemak voor organisatoren aan bod.

6.1. Architectuur van de PoC

6.2. Implementatie van de CTFd-plugin

6.2.1. Technische keuzes

6.2.2. Container lifecycle management

6.2.3. Configuratie van een challenge-container

6.2.4. Gebruikersinterface in CTFd

6.3. Beveiliging in de PoC

6.3.1. Isolatie tussen containers van verschillende teams/gebruikers

6.4. Schaalbaarheid en load balancing

6.5. Automatisatie en gebruiksgemak voor organisatoren

7

Validatie en testing

Dit hoofdstuk beschrijft het testplan en de testresultaten waarmee de correctheid, veiligheid en schaalbaarheid van de PoC worden aangetoond.

7.1. Testplan

7.1.1. Doelstellingen

7.1.2. Testomgeving

7.2. Functionele tests

7.2.1. Aanmaken en beëindigen van containers

7.2.2. Isolatie tussen gebruikers

7.3. Beveiligingstests

7.3.1. Container escape pogingen

7.3.2. Resource exhaustion simulaties

7.4. Load testing

7.4.1. Testopzet

7.4.2. Meetpunten

7.4.3. Resultaten en conclusies

8

Conclusie

- 8.1. Beantwoording van de onderzoeksvragen**
- 8.2. Beperkingen van de PoC**
- 8.3. Aanbevelingen voor productiegebruik**
- 8.4. Mogelijke uitbreidingen**



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

Samenvatting

Dit onderzoeksvoorstel behandelt het dynamisch beheer van Docker-containers voor Capture The Flag (CTF)-events op on-premise infrastructuur. De probleemstelling richt zich op het aanmaken en beëindigen van containers op basis van gebruikersacties. De hoofdonderzoeksvraag onderzoekt hoe dit proces kan worden geautomatiseerd met behulp van CTFd en Docker. Daarbij wordt gewerkt binnen de randvoorwaarden van een volledig self-hosted omgeving, zonder terugkerende licentiekosten en met een voorkeur voor open-source componenten. Via een literatuurstudie en een proof-of-concept worden de beveiligingsaspecten, resource-limieten en communicatie tussen CTFd en Docker onderzocht. Het doel is een schaalbare oplossing te ontwerpen die misbruik helpt voorkomen en de preformantie bewaakt.

A.1. Introductie

Dit onderzoeksvoorstel richt zich op het dynamisch beheer van Docker-omgevingen tijdens Capture The Flag (CTF) events (Innovirtuoso, [z.d.](#)) op on-premise infrastructuur. Het idee voor dit onderzoek komt voort uit de noodzaak om efficiënt en veilig challenges te beheren (Singier, [2025](#)).

A.1.1. Probleemstelling

De centrale probleemstelling luidt: Bestaat er een manier om dynamisch Docker-omgevingbeheer te doen op on-prem infrastructuur tijdens Capture The Flag events?

Het beheren van containers voor challenges vereist een robuust systeem dat kan omgaan met gebruikersacties en variabele belastingen, zonder in te boeten op veiligheid of performance.

Daarnaast is het een vereiste dat de oplossing volledig self-hosted kan worden ingezet. Hiervoor zijn drie belangrijke redenen: flexibiliteit (mogelijkheid tot uitbreidingen en features), kostprijs (het betalen van on-premise infrastructuur versus cloud-diensten is voor veel organisaties een andere financiële flow) en de voorkeur voor open-source oplossingen.

A.1.2. Onderzoeksvraag

De hoofdonderzoeksvraag van dit bachelorproefvoorstel is:

Hoe kunnen we dynamisch onze docker containers gelinkt aan CTFd-challenges beheren, aanmaken en beëindigen op basis van gebruikersacties en ingestelde variabelen?

Om deze hoofdvraag te beantwoorden, worden de volgende deelvragen geformuleerd, opgesplitst in probleem- en oplossingsdomein:

Probleemdomein:

- Welke fundamentele en direct toepasbare beveiligingsmaatregelen ('low-hanging fruit') zijn noodzakelijk om de veiligheid van de Docker-host en -containers te waarborgen?
- Welke specifieke beveiligingsrisico's brengt het dynamisch aanmaken van containers door gebruikers met zich mee op on-premise infrastructuur?
- Wat is de impact van frequente container-creatie en -vernietiging op de performance van de host-server?
- Welke communicatieprotocollen zijn geschikt voor de interactie tussen het CTF-platform en de container-runtime?

Oplossingsdomein (gericht op de Proof-of-Concept):

- Hoe kan een CTFd-plugin ontwikkeld worden die rechtstreeks communiceert met de Docker Engine API zonder zware externe dependencies?
- Op welke manier kunnen resource-limieten (CPU, RAM) en netwerkisolatie technisch worden afgedwongen in de PoC?
- Hoe kan de architectuur zo worden opgezet dat deze volledig self-hosted en modulair uitbreidbaar is?
- Hoe valideren we de stabiliteit van de oplossing onder gesimuleerde zware belasting (load testing)?

A.1.3. Onderzoeksdoelstelling

Het doel is om een werkende oplossing of proof-of-concept te realiseren die aan toont hoe CTFd-challenges dynamisch en veilig beheerd kunnen worden op eigen infrastructuur.

A.2. State-of-the-art

Er is al documentatie beschikbaar over het deployen van challenges binnen CTFd (CTFd, [z.d.](#)). Echter, het veilig en dynamisch beheren van deze containers op eigen infrastructuur brengt specifieke uitdagingen met zich mee.

Beveiliging is een cruciaal aspect; containers moeten geïsoleerd zijn om te voorkomen dat deelnemers dingen kunnen doen op de host of andere containers. Bronnen zoals de Docker Security Cheat Sheet van OWASP (OWASP, [2024](#)) en artikelen over container security (Aikido, [2025](#)) bieden richtlijnen hiervoor.

Daarnaast biedt een overzicht van het Docker-ecosysteem inzicht in de beschikbare tools en technologieën die ingezet kunnen worden (Arxiv, [2024](#)). Het onderzoek zal zich baseren op deze bronnen om een veilige en schaalbare architectuur te ontwerpen.

A.3. Methodologie

Het onderzoek zal bestaan uit een combinatie van literatuurstudie en praktijkgericht onderzoek.

In de eerste fase wordt via literatuurstudie onderzocht welke 'low-hanging fruit' beveiligingsmaatregelen essentieel zijn. De focus ligt hierbij op praktische en effectieve best practices voor container security (zoals resource limits, netwerkisolatie en privilege management) in plaats van diepgaande theoretische beveiligingsmodellen. Daarnaast wordt gekeken naar de Docker Engine API en SDKs voor de integratie.

Vervolgens zal een proof-of-concept (PoC) worden ontwikkeld waarin een CTFd-omgeving wordt opgezet die communiceert met een Docker-daemon. Hierbij wordt gefocust op de implementatie van een API-interface, het instellen van resource limieten en het toepassen van de geïdentificeerde basisbeveiliging.

Tot slot zal de oplossing gevalideerd worden aan de hand van een uitgebreid testplan als onderdeel van de PoC-verificatie. Dit testplan beschrijft de strategie om zowel de stabiliteit als de veiligheid van het systeem te waarborgen. Concreet worden er load tests uitgevoerd om de schaalbaarheid en performance onder zware belasting te meten. Daarnaast worden specifieke beveiligingstests (zoals container escape pogingen en resource exhaustion simulaties) uitgevoerd om te verifiëren of de geïmplementeerde isolatie en restricties effectief zijn tegen misbruik.

A.4. Verwacht resultaat, conclusie

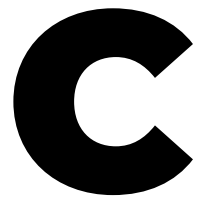
Het verwachte resultaat is een gedocumenteerde en werkende implementatie voor dynamisch containerbeheer binnen CTFd. Dit omvat:

- Een adviesrapport over de te gebruiken architectuur en beveiligingsmaatregelen.
- Een proof-of-concept implementatie.
- Testresultaten die de schaalbaarheid en stabiliteit aantonen.
- Installatie- en beheerinstructies om de reproduceerbaarheid van de PoC te waarborgen.
- Technische documentatie van de ontwikkelde architectuur, zodat derden hierop kunnen verder bouwen.

De meerwaarde van dit onderzoek ligt in het vereenvoudigen van het beheer van CTF-events op eigen hardware, met behoud van veiligheid en performance.

B

Installatie- en beheerinstructies



Configuratiebestanden

D

Testresultaten in detail



Onderzoekslog

Dit appendix documenteert de belangrijkste beslissingen die tijdens het onderzoek werden genomen, met hun motivatie. Het geeft inzicht in het verloop van het onderzoeksproces en de keuzes die de uiteindelijke proof-of-concept hebben gevormd.

Fase 1 – Literatuurstudie en begrippenkader

Beslissing: afbakening van de literatuurstudie

Bij de start van het onderzoek werd beslist de literatuurstudie te richten op vier thema's: CTF-events en het CTFd-platform, Docker en containerisatie, container-beveiliging en load balancing zonder Kubernetes. Dit stelde de lezer in staat de rest van de bachelorproef te volgen zonder voorkennis. Diepgaande theoretische beveiligingsmodellen (zoals formele verificatie of CVE-analyse) werden bewust buiten scope gelaten ten voordele van direct toepasbare best practices (Center for Internet Security, [2023](#); OWASP, [2024](#)).

Beslissing: focus op “low-hanging fruit” beveiliging

Na het bestuderen van de OWASP Docker Security Cheat Sheet (OWASP, [2024](#)) en het CIS Docker Benchmark (Center for Internet Security, [2023](#)) werd gekozen om de focus te leggen op praktisch toepasbare basismaatregelen: capabilities droppen, read-only bestandssysteem, seccomp-profielen, no-new-privileges en resource-limieten. Deze maatregelen bieden een goede balans tussen beveiliging en implementatie-complexiteit voor een CTF-context.

Fase 2 – Analyse van de huidige en gewenste situatie

Beslissing: Kubernetes niet als primaire oplossing

Kubernetes werd vroeg in het onderzoek overwogen als orkestratieplatform. Na analyse werd besloten het niet als primaire oplossing te gebruiken omdat de opera-

tionele complexiteit (clusteropzet, beheer, leercurve) niet in verhouding staat met de schaal van een doorsnee CTF-event. Als alternatief werd gefocust op Docker Swarm (Docker Inc., [z.d.-d](#)) en Traefik (Traefik Labs, [z.d.](#)) als lichtere alternatieven. Dit sluit ook aan bij de randvoorwaarde van eenvoudige inzetbaarheid op eigen infrastructuur.

Beslissing: granulariteit – container per gebruiker of per team

Bij de architectuuranalyse werd de keuze tussen een instantie per gebruiker en een instantie per team bestudeerd. Per gebruiker biedt maximale isolatie maar verhoogt het aantal containers snel. Per team beperkt het resource-gebruik maar verlaagt de isolatiegarantie. Uiteindelijk werd gekozen voor een flexibele aanpak: in teammodus wordt één instantie per team voorzien, bij individuele deelname één per gebruiker. Dit koppelt de granulariteit aan het wedstrijdmodel en vermijdt onnodige containerexplosie.

Beslissing: Docker SDK voor Python vs. directe API-calls

Voor de integratie tussen de CTFd-plugin en de Docker-daemon werden twee opties afgewogen: de officiële Docker SDK voor Python (Docker Inc., [z.d.-c](#)) en directe HTTP-aanroepen naar de Docker Engine API (Docker Inc., [z.d.-b](#)). De SDK biedt een hogere abstractie, minder foutgevoeligheid en is de aanbevolen aanpak voor Python-applicaties. Directe API-calls zouden meer controle geven maar ook meer boilerplate en foutafhandeling vergen. Gekozen werd voor de SDK, tenzij er specifieke beperkingen opduiken tijdens de implementatie.

Beslissing: keuze van reverse proxy

Als toegangspunt voor dynamisch aangemaakte containers werd Traefik (Traefik Labs, [z.d.](#)) verkozen boven Nginx en HAProxy (HAProxy Technologies, [z.d.](#)). De belangrijkste reden is dat Traefik automatisch routes configureert op basis van Docker-labels, wat ideaal is voor dynamisch beheerde containers zonder manuele configuratie-aanpassingen. Nginx en HAProxy vereisen een herstart of dynamische herconfiguratie bij elke nieuwe container, wat de automatisering bemoeilijkt.

Beslissing: plugin-architectuur binnen CTFd

Als implementatiestrategie werd gekozen voor een CTFd-plugin in plaats van een externe service. CTFd ondersteunt plugins via het CTFd/plugins/-mechanisme (CTFd LLC, [z.d.](#)), waarmee nieuwe challenge-types geregistreerd en extra routes toegevoegd kunnen worden. Een interne plugin koppelt gebruikersacties direct aan containeracties zonder extra netwerkverkeer of externe authenticatie.

Beslissing: bestaande plugin CTFd-Whale als referentie, niet als basis

De bestaande CTFd-Whale plugin (Glzjin, [z.d.](#)) werd bestudeerd als referentie voor de architectuuranalyse. Er werd besloten geen fork of uitbreiding te maken, maar een eigen implementatie te schrijven. CTFd-Whale voldoet niet volledig aan de randvoorwaarden van dit onderzoek (volledig self-hosted, open-source, controleerbare beveiligingsinstellingen) en een eigen implementatie biedt meer controle over de beveiligings- en resourceconfiguratie.

Bibliografie

- Aikido. (2025, april 28). *Docker & Kubernetes Container Security Explained*. Verkregen november 28, 2025, van <https://www.aikido.dev/blog/docker-kubernetes-container-security>
- Arxiv. (2024). Navigating the Docker Ecosystem: A Comprehensive Taxonomy and Survey. <https://arxiv.org/pdf/2403.17940.pdf>
- Center for Internet Security. (2023). *CIS Docker Benchmark* (tech. rap.). Center for Internet Security. Verkregen februari 22, 2026, van <https://www.cisecurity.org/benchmark/docker>
- CTFd. (z.d.). *Deploying Challenge Services*. Verkregen november 28, 2025, van <https://docs.ctfd.io/tutorials/challenges/deploying-challenges>
- CTFd LLC. (z.d.). *CTFd: CTFs as you need them*. Verkregen februari 22, 2026, van <https://github.com/CTFd/CTFd>
- Docker Inc. (z.d.-a). *Docker Compose overview*. Verkregen februari 22, 2026, van <https://docs.docker.com/compose/>
- Docker Inc. (z.d.-b). *Docker Engine API*. Verkregen februari 22, 2026, van <https://docs.docker.com/engine/api/>
- Docker Inc. (z.d.-c). *Docker SDK for Python*. Verkregen februari 22, 2026, van <https://docker-py.readthedocs.io/>
- Docker Inc. (z.d.-d). *Swarm mode overview*. Verkregen februari 22, 2026, van <https://docs.docker.com/engine/swarm/>
- Glzjin. (z.d.). *CTFd-Whale: CTFd Plugin for Deploying Standalone Challenges as Docker Containers*. Verkregen februari 22, 2026, van <https://github.com/glzjin/CTFd-Whale>
- Google. (z.d.). *kCTF: Kubernetes-based CTF hosting*. Verkregen februari 22, 2026, van <https://github.com/google/kctf>
- Grafana Labs. (z.d.). *k6 Documentation*. Verkregen februari 22, 2026, van <https://k6.io/docs/>
- HAProxy Technologies. (z.d.). *HAProxy: The Reliable, High Performance TCP/HTTP Load Balancer*. Verkregen februari 22, 2026, van <https://www.haproxy.org/>
- Innovirtuoso. (z.d.). *Capture The Flag (CTF) Explained: How Hacking Competitions Turn You Into a Cyber Pro*. Verkregen december 11, 2025, van <https://innovirtuoso.com/cybersecurity/capture-the-flag-ctf-explained-how-hacking-competitions-turn-you-into-a-cyber-pro/>

- Locust contributors. (z.d.). *Locust: An open source load testing tool*. Verkregen februari 22, 2026, van <https://locust.io/>
- National Institute of Standards and Technology. (z.d.). *National Vulnerability Database*. Verkregen februari 22, 2026, van <https://nvd.nist.gov/>
- OWASP. (2024). *Docker Security Cheat Sheet*. Verkregen november 28, 2025, van https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- redpwn. (z.d.). *rCTF: redpwn's CTF platform*. Verkregen februari 22, 2026, van <https://github.com/redpwn/rctf>
- Singier, L. (2025). Ideeendocument BP.
- Traefik Labs. (z.d.). *Traefik Proxy Documentation*. Verkregen februari 22, 2026, van <https://doc.traefik.io/traefik/>